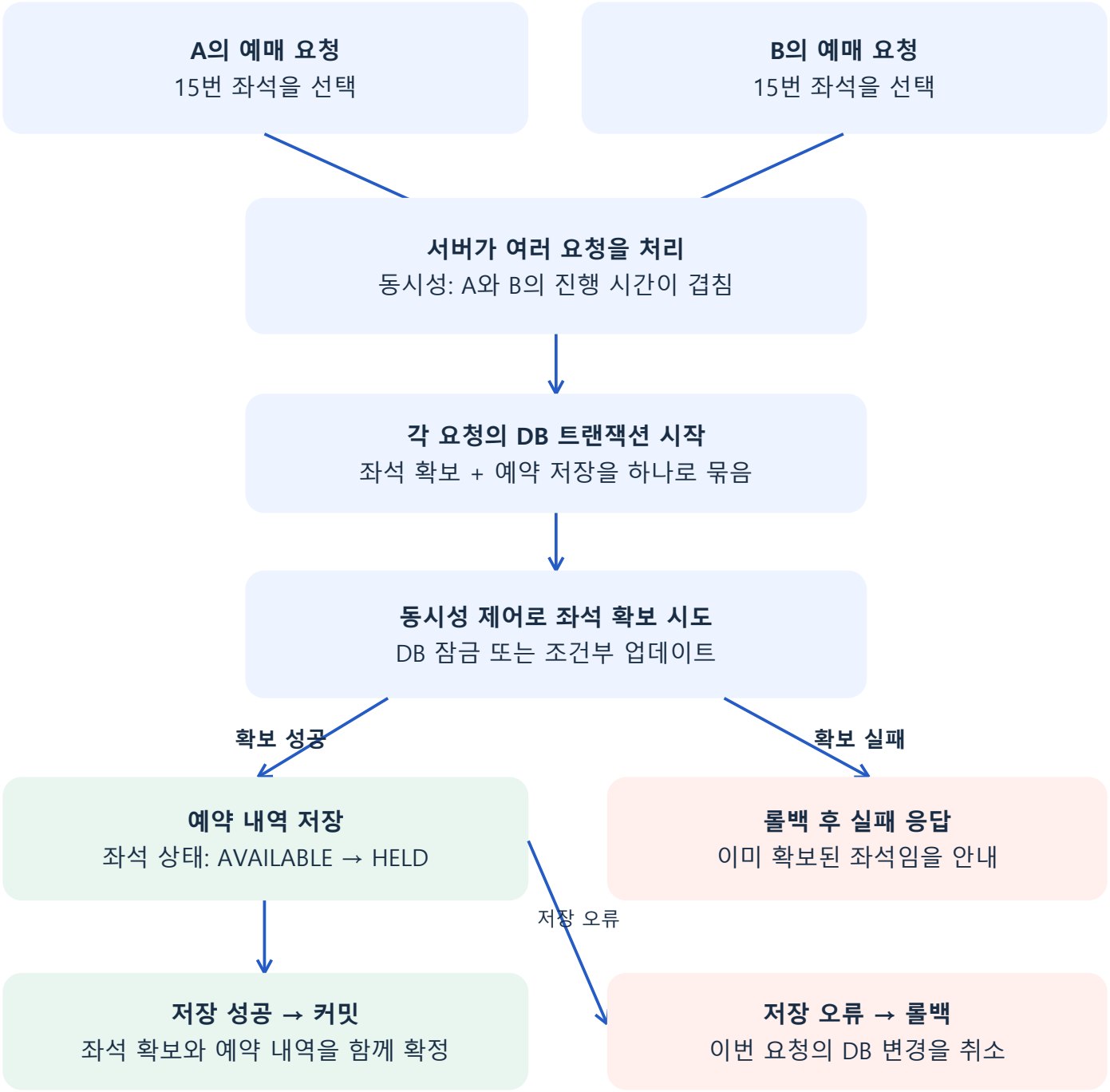
# 티켓 예매의 전체 흐름

예시: A와 B가 마지막 남은 15번 좌석을 거의 동시에 요청한다.




핵심 규칙: 같은 좌석의 유효한 확보는 최대 1건이다.

HELD는 임시 확보다. 커밋했다고 결제가 완료된 것은 아니다.

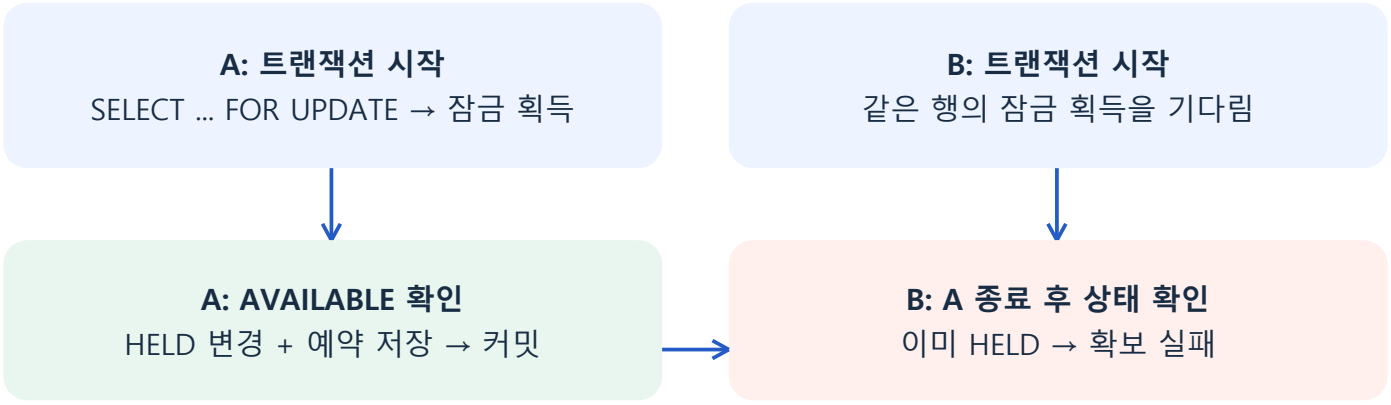
이후 흐름: 결제 진행 → 결제 성공 시 예약 확정 / 실패·만료 시 좌석 해제.

외부 결제는 DB 롤백으로 취소되지 않으므로 별도 취소 처리가 필요하다.

# 중복 예매를 막는 두 가지 방법

두 방법 모두 DB에서 규칙을 지킨다. 비동기 처리만으로 중복 예매를 막을 수 없다.

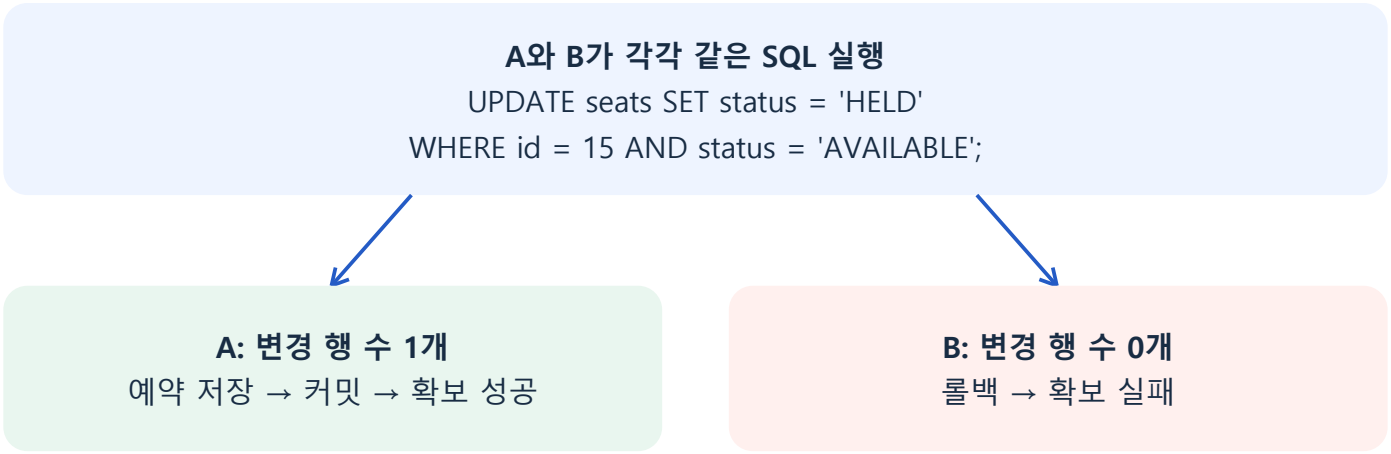
## ① DB 잠금: 먼저 잠근 요청의 처리가 끝날 때까지 대기




잠금은 보통 커밋·롤백까지 유지된다. 확인과 변경을 같은 트랜잭션에 둔다.

일반 SELECT까지 모두 막는 것은 아니다. 모든 예약 경로가 보호 규칙을 따라야 한다.

## ② 조건부 업데이트: 아직 AVAILABLE일 때만 바꾸기




예시는 A가 먼저 변경하고 커밋한 경우다. A가 롤백하면 B가 성공할 수도 있다.

중요: 사전 조회 결과가 아니라, UPDATE의 변경 행 수로 확보 성공을 판단한다.

조건부 UPDATE도 내부적으로 DB의 잠금 등 동시성 처리 기능을 사용한다.

**선택 기준**

복잡한 조회·판단에는 명시적 잠금 / 단순한 상태 전환에는 조건부 UPDATE

DB·격리 수준에 따라 경합 오류가 날 수 있으며, 실패 처리 또는 정해진 재시도가 필요하다.

# 개념을 한 장으로 연결하기

비동기·동시성·병렬성은 처리 방식에 관한 개념이며, 데이터 보호는 별도로 설계한다.

**비동기 | 기다리는 시간을 활용**
A가 DB 응답을 기다리는 동안 서버가 B의 요청을 진행한다.

**동시성 | 여러 작업의 진행 시간이 겹침**
A가 끝나기 전에 B도 시작된다. 한 작업자가 번갈아 처리해도 성립한다.

**병렬성 | 같은 순간에 실제 실행**
서로 다른 CPU 코어에서 A와 B의 작업을 실제로 함께 실행한다.

**동시성 제어 | 공유 데이터의 규칙 보호**
A와 B가 같은 좌석을 요청해도 한 명만 확보하도록 DB에서 조정한다.

## 트랜잭션 · 커밋 · 롤백

**트랜잭션**
좌석 상태 변경과 예약 내역 저장을 전부 성공하거나 전부 취소하도록 묶음

**커밋 = 변경 확정**
HELD + 예약 내역을 함께 반영

**롤백 = 미확정 변경 취소**
이번 트랜잭션의 변경을 되돌림

트랜잭션으로 묶기만 해서는 중복 예매가 자동으로 방지되지 않는다.

트랜잭션 안에서 잠금 또는 조건부 업데이트를 올바르게 사용해야 한다.

결제 대기 동안 DB 잠금을 유지하지 않고, 임시 확보 상태와 만료를 별도로 관리한다.